

UniSim-SLAM: Feed-Forward SLAM with Unified Sim(3) Optimization

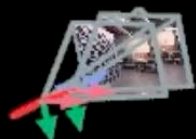
Inha Lee¹, Dongjae Jeong¹, Junhee Lee¹, Kyungdon Joo^{1,†}

ECCV 2026

발표자: 송우석

Demo

x5



Intro

- 최근 연구들에 따르면, geometric foundation models이 visual SLAM 시스템의 구조를 바꾸고 있음 (e.g. DUS3R, VGGT)
 - feed-forward models을 특정 view subsets에 적용하기 시작함
 - image pairs
 - multi-view image clips
 - 각 추론된 결과를 local 좌표계에서 정의한 local reconstruction으로 하여 global coordinate system에서 정렬
- 문제점
 - geometric estimates이 input view configuration에 제약이 있고, 예측이 장기 시퀀스로 이어지면서 geometric inconsistencies를 유발

⇒ 따라서, global trajectory는 다수의 각 고유의 좌표계에서 정의된 configuration-dependent local reconstructions을 정렬함으로써 만들어져야 함

Visual SLAM

- feature-based methods
 - SfM pipeline을 따르며, keypoint matching을 이용하여 triangulation, PnP 수행
- direct methods
 - 보통 frame별 depth estimation과 함께 photometric residuals을 최소화하여 camera pose를 최적화함
→ 하지만 이들은 camera calibration과 challenging visual conditions에 민감
- learning-based SLAM
 - deep neural network를 frontend나 scene representation에 통합하여 강건성을 높이고, densify mapping을 이룸
- Neural implicit mapping
 - camera poses와 implicit scene representation을 online으로 동시 최적화
- 3DGS
 - poses와 Gaussian primitives를 동시 최적화

⇒ 그럼에도 불구하고, 대부분의 learning-based 및 neural mapping 방법론들은 정확한 intrinsics에 의존하거나 실시간 이용을 제한하는 무거운 연산량을 지님

Feed-Forward Visual SLAM

- 3D geometric foundation model을 이용해 geometry와 camera parameter를 feed-forward 방식으로 예측하게 함
 - handcrafted correspondences와 heavy per-sequence optimization에 대한 의존을 줄임
- pairwise feed-forward models
 - uncalibrated image pair 를 입력받아 3차원 구조와 dense correspondence를 예측
 - retrieval-based view graphs와 global alignment를 통해 unconstrained collections으로 확장 가능
 - + Memory or state-based models, multi-view foundation models 등으로 확장 가능
- feed-forward visual SLAM 방법론 등장
 - two-view pipeline (e.g. MAST3R-SLAM, ViSTA-SLAM)
 - pairwise prediction과 lightweight global optimization을 수행
 - multi-view pipeline (e.g. VGGT-SLAM)
 - multi-view reconstruction backbone으로 만들어진 submap들을 점진적으로 정렬 및 전역적으로 최적화하는데 집중

⇒ 하지만 대부분의 feed-forward SLAM 시스템들은 two-view temporal constraints나 multi-view submap constraints 중에 하나만 이용함

Inference

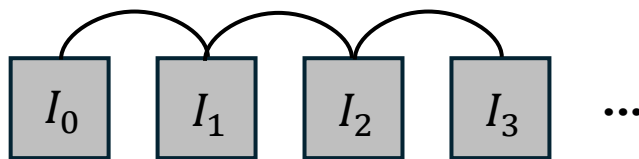
- Two-View (e.g. DUST3R)

- 연산이 가벼움

- 새로운 프레임을 받자마자 동작하여 자연스럽게 temporal connectivity를 유지하면서 low latency를 보임

→ 하지만 제한된 geometric constraints 때문에, 시간에 따른 drift 축적되기 쉬움

→ long-term global consistency를 유지하기에 불충분함



- Multi-View (e.g. VGGT)

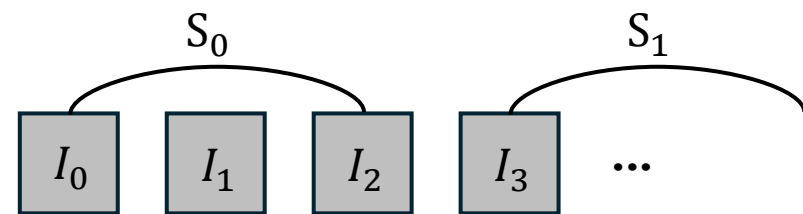
- 풍부한 geometric constraints을 이용하지만 추론 전에 고정된 개수의 프레임들을 누적시키는 것을 필요로 함 (i.e., submap)

→ per-frame updates를 비실용적으로 만듦

- submap간의 관계만 이용한 보정(refinement)는 trajectory를 따라 correction을 전파하기 위해 이웃하는 submap들간의 겹침(overlap)이 필요함

- 특히, 겹침이 없을 때, geometric corrections은 효과적으로 전파될 수 없음

→ global consistency가 제한됨



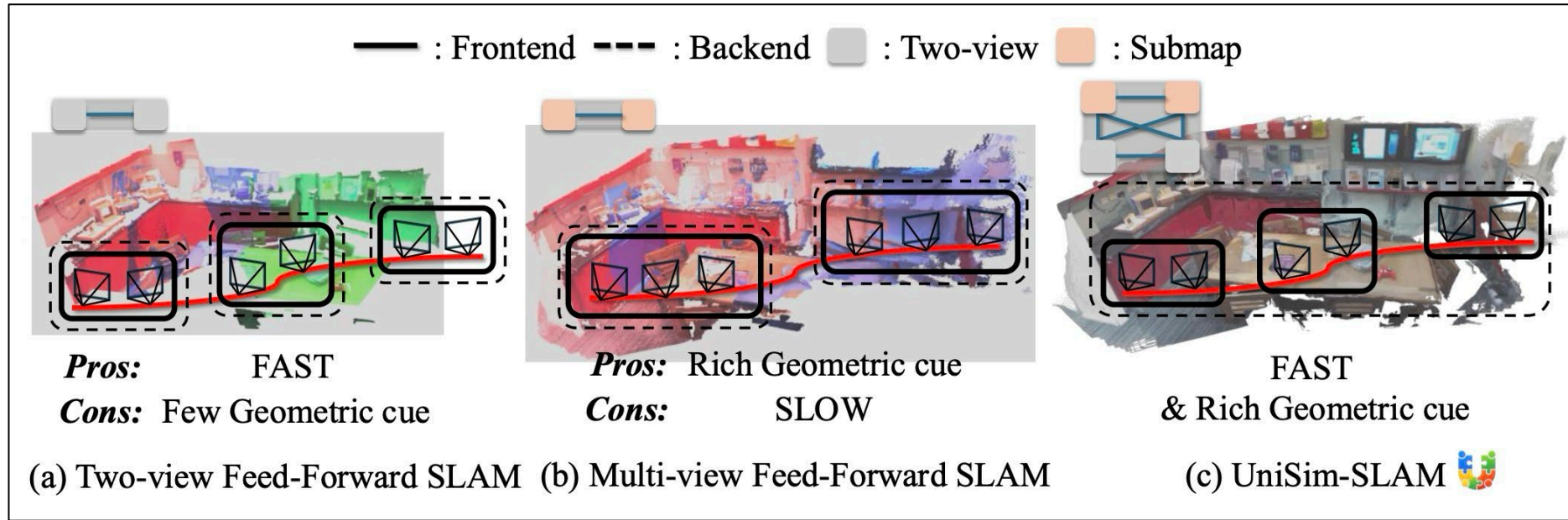
⇒ low-latency tracking과 long-term geometric consistency간의 trade-off를 다루기 위해 고전(classic) SLAM 시스템의 구조 원리를 재방문

Classical SLAM System

- 앞 문제와 비슷한 efficiency-consistency trade-off를 완화함
 - frontend: lightweight two-view odometry
 - backend: intermittent(간헐적) globally consistent refinement
 - 이는 two-view inference의 temporal consistency를 통해 강건한 graph connectivity를 보장하고, 동시에 multi-view submap의 rich geometric constraints를 이용함
 - 하지만 기존 feed-forward 방법들은 보통 pairwise two-view alignment나 submap-to-submap registration 중에 하나만 수행함
- two-view와 multi-view predictions을 naïvely하게 합치는 것은 non-trivial한데, 이는 해당 prediction들의 scale과 reference frame이 일치하지 않는 heterogeneous local coordinate systems에 속하기 때문
 - 따라서, 새로운 factor graph formulation은 이러한 heterogeneous constraints를 single unified framework 내에서 동시에 최적화 하는 것이 필요함

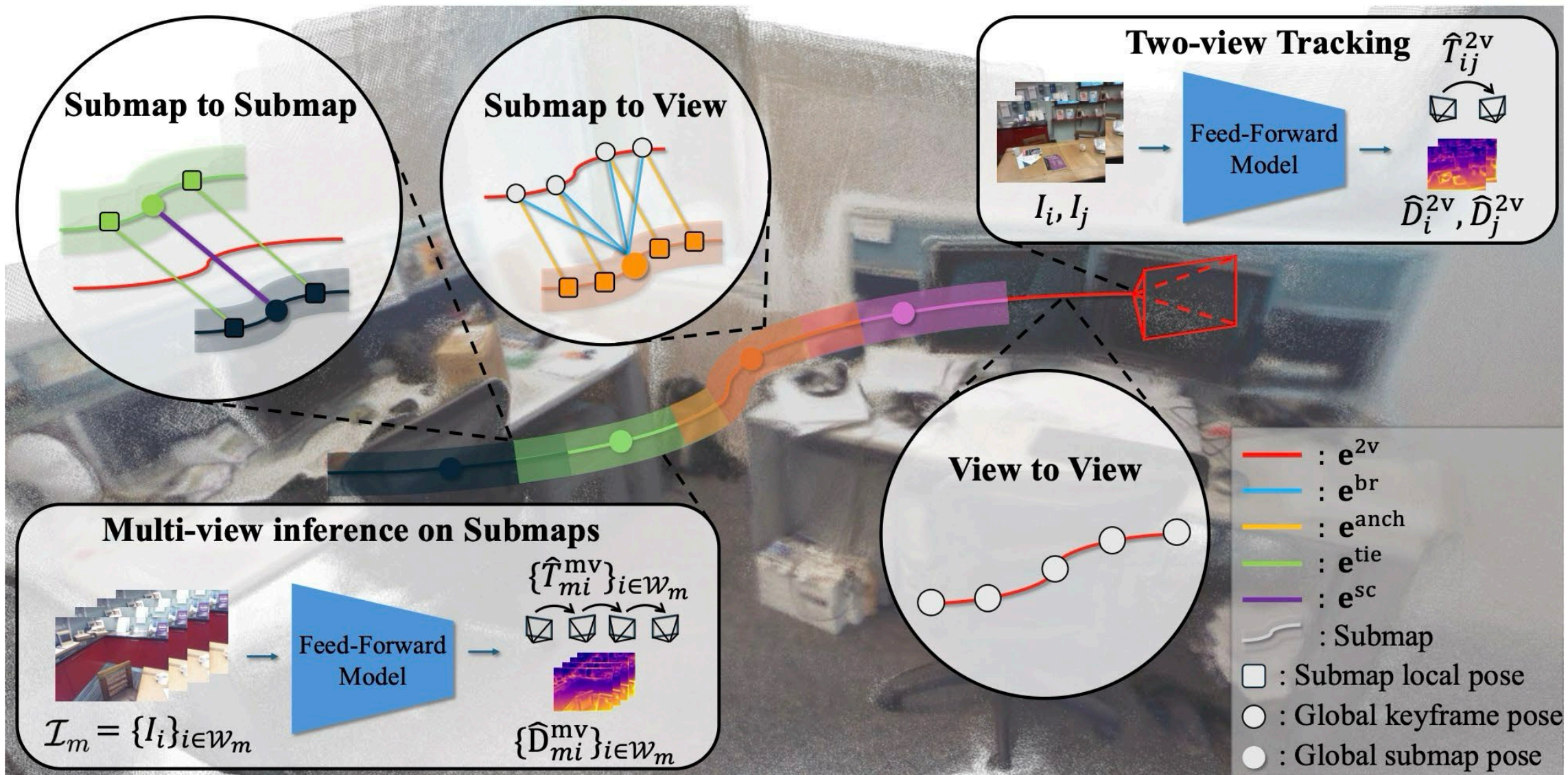
⇒ UniSim-SLAM의 등장

UniSim-SLAM



- two-view와 multi-view inference를 하나의 $Sim(3)$ factor graph 내에서 통합하는 feed-forward SLAM system (둘 다 활용)
- 서로 다른 feed-forward inference 방법에서 얻은 geometric prediction을 heterogeneous local 좌표계 시스템에서 상호보완적인 constraints로 통합하기 위해 unified $Sim(3)$ factor graph에서 동시에 최적화
- feed-forward SLAM에서 효율성-일관성 trade-off를 해결하고 low-latency tracking과 long-term geometric stability를 달성하여 표준 SLAM benchmarks에서 SoTA급 성능을 보여줌

UniSim-SLAM



Two-View Tracking on Keyframes

- frontend는 low latency와 temporal consistency를 보장

$\{T_i\}$: global keyframe poses

$\{S_m\}$: submap poses

keyframe I_i 에서 I_j 로의 상대 변환 $\in Sim(3)$

(I_i, I_j) : keyframe pair (with $j = i + 1$)

$$\{\hat{D}_i^{2v}, \hat{D}_j^{2v}, \hat{T}_{ij}^{2v}\} = f_{2v}(I_i, I_j), \quad (1)$$

keyframe I_i 의 predicted depth map

two-view feed-forward model

- 첫 번째 keyframe I_0 이 전역 좌표계의 origin (global reference frame)
- 순차적으로 global pose를 추정하고, 결과로 나온 궤적이 초기 추정치가 됨

$$T_j = T_i \hat{T}_{ij}^{2v}, \quad (2) \quad \Rightarrow \text{나중에 backend에서 multi-view submap constraints로 보정}$$

Multi-view Submap Integration

- backend의 목표
 - global pose $\{T_i\}$ 에 누적된 drift를 보정하는 것
 - 이를 위해 multi-view submaps을 형성하여 주기적으로 global pose graph에 geometrically rich constraints를 끼워 넣음

- submap

$$\mathcal{W}_m \subset \{1, \dots, N\}$$

keyframe index

- corresponding keyframe sequence

$$\mathcal{I}_m = \{I_i\}_{i \in \mathcal{W}_m}$$

- feed forward multi-view model

$$\{\hat{D}_{mi}^{mv}, \hat{T}_{mi}^{mv}, \}_{i \in \mathcal{W}_m} = f_{mv}(\mathcal{I}_m), \quad (3)$$

multi-view feed-forward model

submap-local pose (submap origin keyframe을 기준으로 각 keyframe i 의 변환을 의미)

Multi-view Submap Integration

- submap origin
 - $\mathcal{W}_m$ 의 center keyframe을 submap의 origin keyframe으로 정의
- submap pose
 - submap에서 만들어지는 local reconstruction을 global trajectory에 통합하기 위해, submap pose $S_m \in Sim(3)$ 을 제안하며 이는 m번째 submap 좌표계를 global frame으로 변환함

$$T_i \approx S_m \hat{T}_{mi}^{mv}. \quad (4)$$

- problem
 - multi-view prediction이 view-set dependent하기 때문에, 동일한 keyframe은 submap에 따라 scale과 pose가 달라질 수 있음
 - 따라서 submap-local pose가 globally consistent measurement가 되기 위해서는 submap pose $\{S_m\}$ 을 명시적으로 초기화 해줘야 함

$$\hat{T}_{mi}^{mv} = \mathbf{I}$$

여기서 I_i 는 submap의 origin keyframe을 의미

Multi-view Submap Integration

- two-view inference에서 얻은 global pose

$$T_i = \begin{bmatrix} s_i R_i & t_i \\ 0^T & 1 \end{bmatrix}, \quad \begin{array}{l} \bullet R_i \in \text{SO}(3) \\ \bullet t_i \in \mathbb{R}^3 \\ \bullet s_i \in \mathbb{R}^+ \end{array}$$

- two-view와 multi-view prediction간의 relative scale ambiguity를 해결하기 위해, depth statistics을 사용한 relative scale을 추정

- $s_{mi}^{\text{rel}} = \text{median}(\hat{D}_{mi}^{\text{mv}} / \hat{D}_i^{2\text{v}})$

→ depth statistics

- $s_m^{(0)} = s_i / s_{mi}^{\text{rel}}$

→ 초기 submap scale (multi-view prediction은 unit scale로 표현되기 때문)

Multi-view Submap Integration

- submap pose

$$S_m^{(0)} = \begin{bmatrix} s_m^{(0)} R_i & \frac{t_i}{s_{mi}^{\text{rel}}} \\ 0^\top & 1 \end{bmatrix}.$$

global frame과 submap frame간의 similarity transformation
을 보존하기 위해 동일한 relative factor로 scaling

(6)

- submap의 scale-consistent embedding을 global trajectory에 제공하고, 그 후에 T_i 와 S_m 둘 다 unified optimization으로 동시에 최적화됨
- 하지만 two-view와 multi-view constraints가 동시에 적용될 때, 다수의 Sim(3) 관계가 하나의 submap에서 공존하게 됨
 - 각 keyframe은 global pose T_i 와 associate되고, submap은 Sim(3) 변수 S_m 과 associate

⇒ 따라서, 초기화 단계에서 하나의 relative scale constraint를 강제하는 것은 global consistency를 보장하기에 불충분함

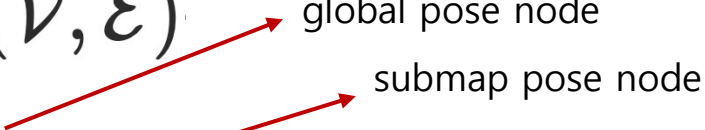
⇒ multi-level factor graph formulation 등장

Unified Sim(3) Pose Graph

- two-view, multi-view feed-forward inferences는 서로 다른 좌표계에서 독립적인 scale로 Sim(3) predictions을 생성
- 전통적인 pose graph로 이러한 heterogenous constraints를 직접적으로 강제하는 것은 호환되지 않는 similarity relations와 불안정한 최적화를 초래할 수 있음

→ 이러한 구조적 비일관성을 극복하기 위해, Sim(3) manifold에서 직접적으로 unified multi-level pose graph를 고안했으며, 이는 pairwise나 submap constraints로만 최적화하는 이전 파이프라인들과 달리, 모든 heterogeneous Sim(3) relations를 하나의 similarity-consistent graph에 모델링함

- Unified Graph

$$\mathcal{G} = (\mathcal{V}, \mathcal{E})$$


global pose node

submap pose node

$$\mathcal{V} = \mathcal{V}_T \cup \mathcal{V}_S, \quad \mathcal{V}_T = \{T_i \in Sim(3)\}, \quad \mathcal{V}_S = \{S_m \in Sim(3)\}$$

$$\mathcal{E} = \mathcal{E}^{\text{temp}} \cup \mathcal{E}^{v2s} \cup \mathcal{E}^{s2s}$$

edge set은 3개의 계층 구조로 나뉨

Unified Sim(3) Pose Graph

- edge set

$$\mathcal{E} = \mathcal{E}^{\text{temp}} \cup \mathcal{E}^{v2s} \cup \mathcal{E}^{s2s}$$

$\mathcal{E}^{\text{temp}}$: temporal edges (View-to-View Edges)

→ 연결성을 보장하기 위해 연속적인 global poses를 연결

$\mathcal{E}^{v2s}$: View-to-Submap edges

→ T_i 를 S_m 과 연결하여 local predictions을 정렬하고, relative scales을 제한

$\mathcal{E}^{s2s}$: Submap-to-Submap edges

→ 직접적으로 overlapping submaps을 연결하여 similarity consistency를 강제하고 scale drift를 예방

⇒ 각각의 edge set에 걸쳐 geometric relationship을 인코딩

Unified Sim(3) Pose Graph

- View-to-View Edges

$$\mathcal{E}^{\text{temp}} = \{(i, j) | j = i + 1\}$$

- 각 $(i, j) \in \mathcal{E}^{\text{temp}}$ 에 대해 two-view feed-forward model은 relative Sim(3) measurement $\hat{T}_{ij}^{2v}$ 를 제공
- global pose와 pairwise feed-forward prediction간의 agreement를 enforce하는 temporal consistency constraints
적용

$$T_i^{-1}T_j \approx \hat{T}_{ij}^{2v}$$

- two-view residual

$$\mathbf{e}_{ij}^{2v} = \log \left((\hat{T}_{ij}^{2v})^{-1} (T_i^{-1}T_j) \right)$$


Unified Sim(3) Pose Graph

- View-to-Submap Edges

$$\mathcal{E}^{v2s} = \{(m, i) | i \in W_m\}$$

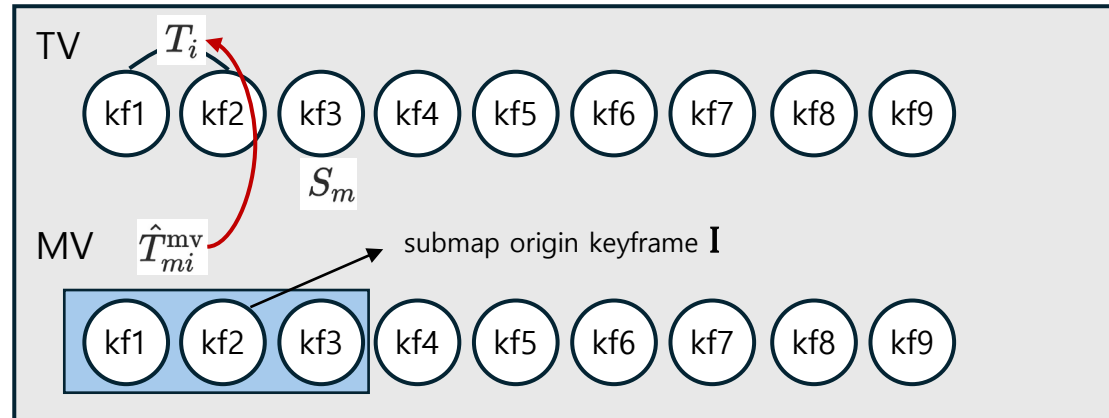
- global pose node T_i 와 대응하는 submap pose node S_m 을 연결
- 각 $(m, i) \in \mathcal{E}^{v2s}$ 에 대해, submap m 은 submap-local pose $\hat{T}_{mi}^{mv}$ 를 가짐

- 2개의 residual terms

- pose alignment (bridge residual)

$$T_i \approx S_m \hat{T}_{mi}^{mv}$$

$$\mathbf{e}_{mi}^{br} = \log \left((\hat{T}_{mi}^{mv})^{-1} S_m^{-1} T_i \right)$$



⇒ 하지만, pose alignment만으로는 global trajectory와 submap 좌표계간의 relative scale에 제약을 주지 못함

- scale anchoring (anchor residual)

$$\mathbf{e}_{mi}^{anch} = \log s_i - \log s_m - \log \left(\text{median}(\hat{D}_{mi}^{mv} / \hat{D}_i^{2v}) \right)$$

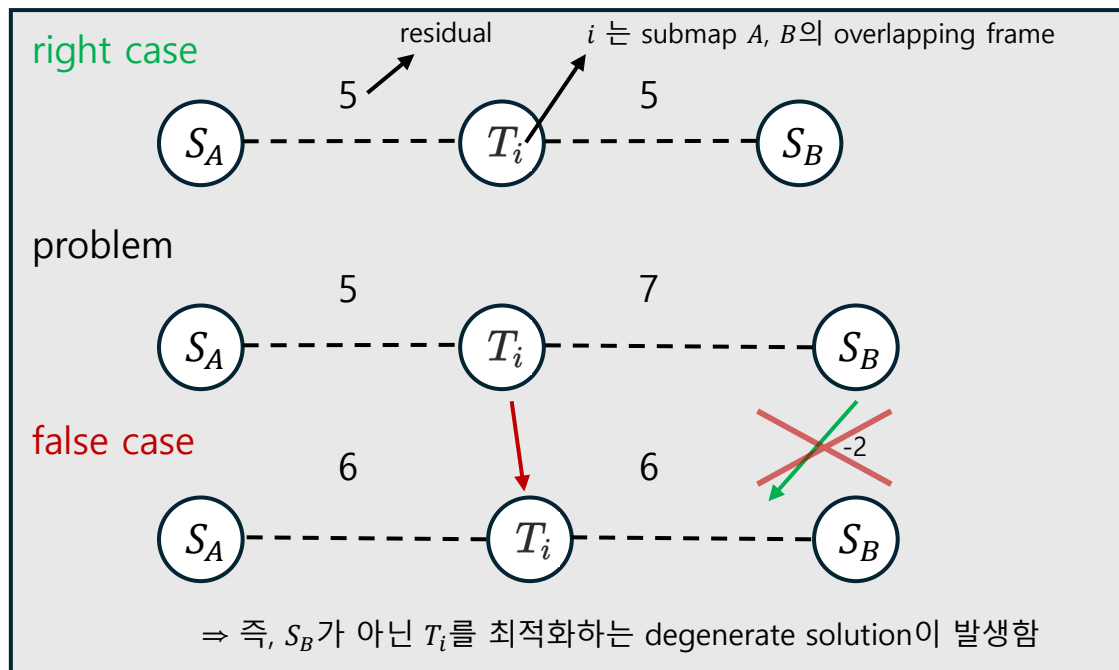
$\hat{D}_i^{2v}$: predicted two-view depth
 $\hat{D}_{mi}^{mv}$: predicted multi-view depth

Unified Sim(3) Pose Graph

- View-to-Submap Edges 만으로는 global view nodes를 통해 submap들간의 간접적인 정렬을 가능하게 하지만, 해당 coupling은 중간 view node들을 교체하는 식으로 보완될 수도 있음
- bridge residual

$$\mathbf{e}_{mi}^{\text{br}} = \log \left((\hat{T}_{mi}^{\text{mv}})^{-1} S_m^{-1} T_i \right)$$

- GT가 아님 (Two-View의 초기 추정치)
→ 따라서 S_m 이 아니라 T_i 를



따라서 Submap들간에 더 엄격한 제약이 필요함

⇒ Submap-to-Submap Edges 등장

Unified Sim(3) Pose Graph

- Submap-to-Submap Edges

$$\mathcal{E}^{s2s} = \{(m, n) | \mathcal{W}_m \cap \mathcal{W}_n \neq \emptyset\}$$

$$\mathcal{V}_{mn} = \mathcal{W}_m \cap \mathcal{W}_n$$

→ 서로 keyframe window가 겹치는 submap들만 연결

→ submap m과 n간에 공유되는 keyframe들

- 2개의 residual terms

- pose consistency constraint (tie residual)

- geometric consistency를 만족하려면 multi-view inference로 submap m 과 n 각각에 대해 독립적으로 출력한 local pose predictions $\hat{T}_{mi}^{mv}$ 와 $\hat{T}_{ni}^{mv}$ 에 대해 submap 좌표계에서 공유되는 view가 동일한 global pose를 내놓아야 함

$$S_m \hat{T}_{mi}^{mv} \approx S_n \hat{T}_{ni}^{mv}$$

$$\mathbf{e}_{mni}^{\text{tie}} = \log \left((S_m \hat{T}_{mi}^{mv})^{-1} (S_n \hat{T}_{ni}^{mv}) \right)$$

- scale consistency constraint (scale residual)

$$\hat{s}_{mn,i} = \text{median}(\hat{D}_{mi}^{mv} / \hat{D}_{ni}^{mv}) \quad \hat{s}_{mn} = \text{median}_{i \in \mathcal{V}_{mn}} \hat{s}_{mn,i}$$

$$\mathbf{e}_{mn}^{\text{sc}} = \log s_n - \log s_m - \log \hat{s}_{mn}$$

⇒ 이러한 direct constraint는 중간 view들의 pose를 조정하여 submap 정렬을 이루는 degenerate solutions를 예방

Optimization

- View-to-View Edges
 - **two-view residual**: 인접 keyframe간에 실제 변환과 two-view feed-forward model로 추정된 변환이 동일해야 한다는 temporal consistency constraint를 줌
- View-to-Submap Edges
 - **bridge residual**: submap origin을 기준으로 정렬(submap-local pose)되어 있는 keyframe을 global pose로 가져오는 변환 (S_m)을 constraint로 줌
 - **anchor residual**: relative scale constraint를 주기 위해 per-view depth statistics을 이용
- Submap-to-Submap Edges
 - **tie residual**: 두 submap이 공유하는 overlapping frame에 대해 예측한 local pose prediction을 global pose로 가져왔을 때 동일한 global pose를 가져야 한다는 constraint를 줌
 - **scale residual**: overlapping frame의 scale이 동일하다는 것을 이용한 scale consistency constraint를 줌
- **Optimization Function**

huber loss function

$$\arg \min_{\{T_i\}, \{S_m\}} \sum_{(i,j) \in \mathcal{E}^{\text{temp}}} \rho(\|\mathbf{e}_{ij}^{2v}\|) + \sum_{(m,i) \in \mathcal{E}^{\text{v2s}}} (\rho(\|\mathbf{e}_{mi}^{\text{br}}\|) + \rho(\|\mathbf{e}_{mi}^{\text{anch}}\|)) + \sum_{(m,n) \in \mathcal{E}^{\text{s2s}}} \left(\sum_{i \in \mathcal{V}_{mn}} \rho(\|\mathbf{e}_{mni}^{\text{tie}}\|) + \rho(\|\mathbf{e}_{mn}^{\text{sc}}\|) \right),$$

- Loop Closure는 VGGT-SLAM 거를 갖다 씀

Experiment

Method	office0	office1	office2	office3	office4	room0	room1	room2	Avg.
MASt3R-SLAM* [26]	0.056	0.056	0.079	0.056	0.059	0.102	0.108	0.063	0.072
VGGT-SLAM [22]	0.056	0.034	0.043	0.057	0.038	0.034	0.037	0.036	0.042
ViSTA-SLAM [47]	0.074	0.193	0.118	0.048	0.130	0.069	0.093	0.136	0.108
Ours	0.068	0.018	0.022	0.020	0.028	0.023	0.021	0.028	0.029

- Camera Pose Estimation

planar structure가 대부분

Replica

Method	360	desk	desk2	floor	plant	room	rpy	teddy	xyz	Avg
Calib.	ORB-SLAM3 [3]	×	0.017	0.210	×	0.034	×	×	×	N/A
	DeepV2D [35]	0.243	0.166	0.379	1.653	0.203	0.246	0.105	0.316	0.375
	DeepFactors [4]	0.159	0.170	0.253	0.169	0.305	0.364	0.043	0.601	0.233
	DPV-SLAM [19]	0.112	0.018	0.029	0.057	0.021	0.330	0.030	0.084	0.076
	DPV-SLAM++ [19]	0.132	0.018	0.029	0.050	0.022	0.096	0.032	0.098	0.054
	GO-SLAM [49]	0.089	0.016	0.028	0.025	0.026	0.052	0.019	0.048	0.035
	DROID-SLAM [36]	0.111	0.018	0.042	0.021	0.016	0.049	0.026	0.048	0.038
	MASt3R-SLAM [26]	0.049	0.016	0.024	0.025	0.020	0.061	0.027	0.041	0.030
UnCalib.	CUT3R [40]	0.174	0.592	0.546	0.662	0.467	0.911	0.051	0.845	0.486
	SLAM3R [21]	0.211	0.861	0.967	0.790	0.755	1.013	0.063	0.986	0.648
	MASt3R-SLAM* [26]	0.070	0.032	0.055	0.056	0.035	0.118	0.041	0.116	0.060
	VGGT-SLAM [22]	0.063	0.031	0.048	0.152	0.023	0.133	0.038	0.039	0.061
	ViSTA-SLAM [47]	0.104	0.030	0.030	0.070	0.052	0.067	0.023	0.080	0.052
	UniSim-SLAM	0.067	0.018	0.022	0.034	0.029	0.056	0.021	0.031	0.013

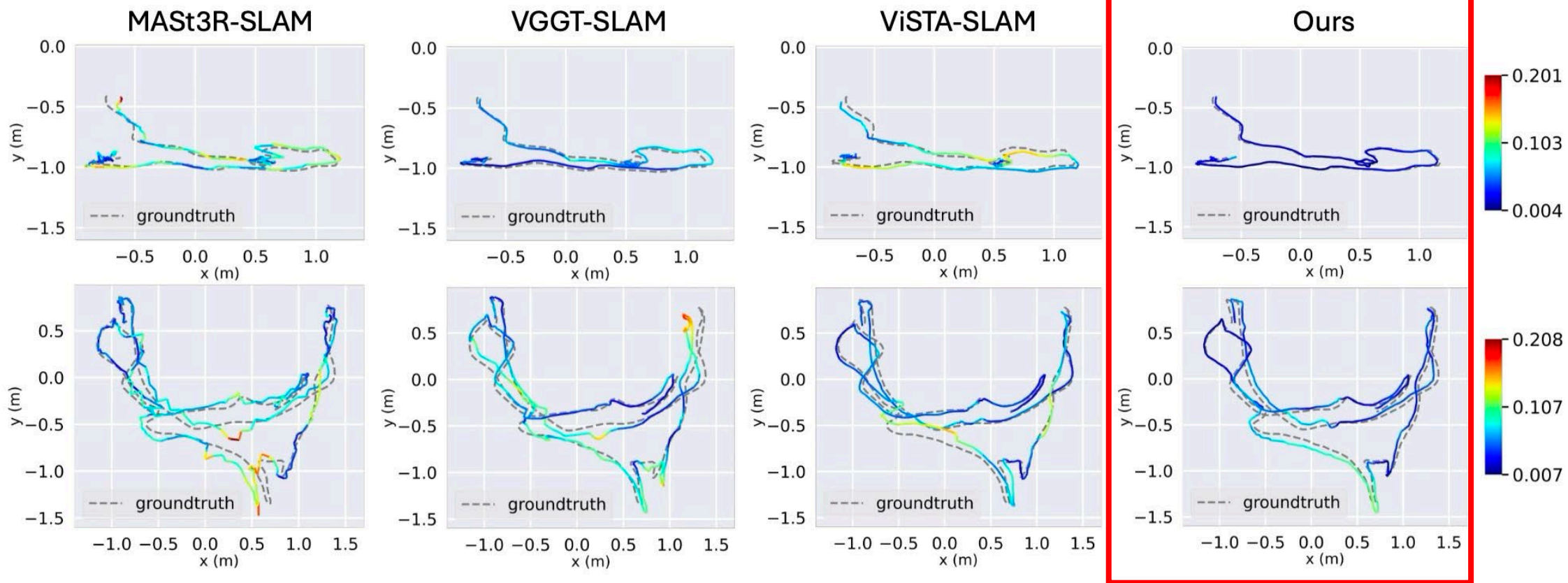
TUM RGB-D

	Method	chess	fire	heads	office	pumpkin	kitchen	stairs	Avg.
Calib.	DROID-SLAM [36]	0.018	0.027	0.021	0.041	0.025	0.016	0.017	0.024
	MASt3R-SLAM [26]	0.082	0.030	0.024	0.052	0.050	0.044	0.027	0.044
Uncalib.	CUT3R [40]	0.514	0.110	0.197	0.430	0.346	0.202	0.385	0.312
	SLAM3R [21]	0.131	0.044	0.040	0.058	0.100	0.064	0.116	0.079
	MASt3R-SLAM [26]	0.090	0.058	0.039	0.072	0.084	0.062	0.071	0.068
	VGGT-SLAM [22]	0.039	0.024	0.041	0.032	0.050	0.034	0.042	0.037
	ViSTA-SLAM [47]	0.075	0.035	0.030	0.064	0.065	0.041	0.036	0.049
	UniSim-SLAM	0.017	0.018	0.026	0.024	0.022	0.016	0.019	0.020

7-Scenes

Experiment

- Camera Pose Estimation



7-Scenes

Experiment

- 3D Reconstruction

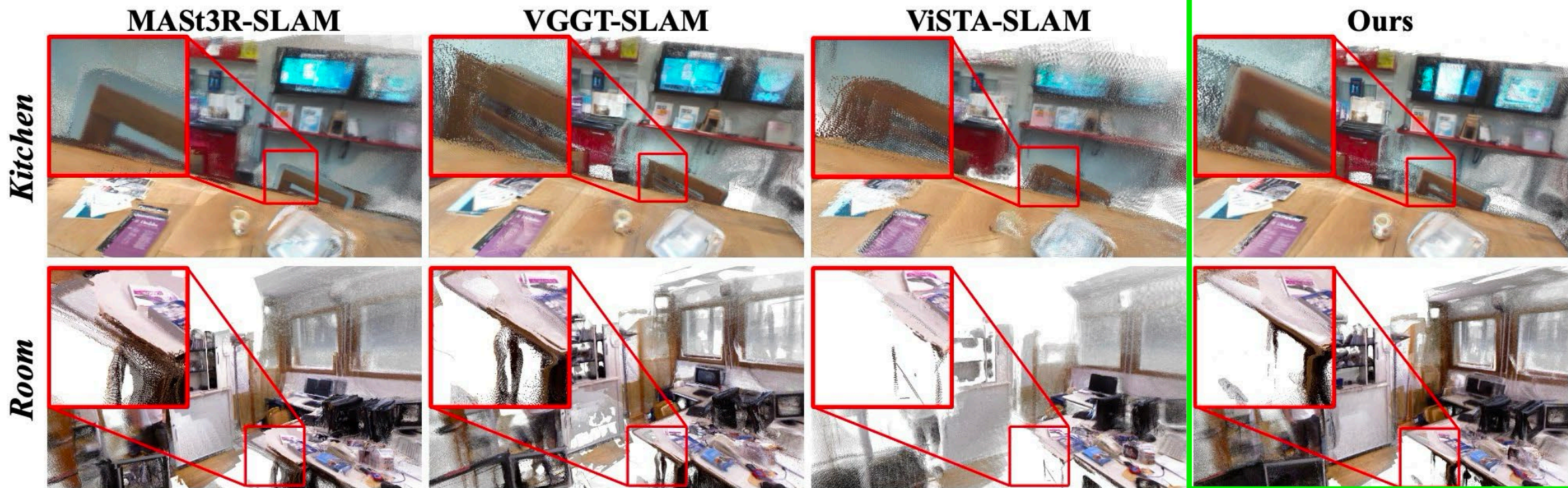
Method		Acc. ↓	Comp. ↓	Chamfer ↓
<i>Calib.</i>	DROID-SLAM [36]	0.111	0.049	0.080
	MASt3R-SLAM [26]	0.064	0.068	0.066
<i>Uncalib.</i>	Spann3R @5 [38]	0.095	0.041	0.068
	CUT3R [40]	0.107	0.059	0.083
	SLAM3R [21]	0.069	0.153	0.111
	MASt3R-SLAM [26]	0.054	0.048	0.051
	VGGT-SLAM [22]	0.039	0.051	0.045
	ViSTA-SLAM [47]	0.041	0.056	0.049
	UniSim-SLAM	0.035	0.046	0.041

7-Scenes

- UniSim-SLAM은 submap간에 overlapping frame을 활용하기 때문에, naïve하게 모든 reconstructed point를 합치는 것은 point density를 증가시켜 인위적인 completion metric 성능 향상을 시킬 수 있음
- 해당 bias를 없애기 위해 하위 25%의 confidence score를 가지는 point들을 제거

Experiment

- 3D Reconstruction



7-Scenes

Experiment

- Latency

(c) Latency comparison on 7-Scenes.

Method	Frontend	Latency [ms]↓	ATE↓
MASt3R-SLAM [26]	MASt3R [17]	90	0.068
VGGT-SLAM [22]	VGGT [39]	3410	0.037
ViSTA-SLAM [47]	STA [47]	35	0.049
Ours + STA [47]	STA [47]	35	0.027
Ours	VGGT [39]	197	0.020

- 여기서 latency는 최소한으로 요구되는 입력 프레임을 받았을 때부터 frontend에서 camera pose estimate를 수행하는데까지 걸린 시간을 의미 (근데 two-view를 처리하는거여서 짧은게 당연함...)
- heterogeneous feed-forward models의 prediction을 통합하면서, multi-level factor graph가 효과적으로 Sim(3) 보정을 수행할 수 있다는 것을 보여줌

- SLAM 시스템에서 입력 frame이 들어 올 때, camera pose 추정은 low latency를 가정함. 따라서 tracking동안에 method별 latency를 분석함
- VGGT를 frontend로 사용했을 때 동일한 multi-view method인 VGGT-SLAM 대비 197ms라는 훨씬 적은 latency로 더 좋은 성능을 보임 (하지만 two-view method인 MASt3R-SLAM 대비 latency가 더 높았음)
- 추가적으로 frontend를 더 가벼운 STA로 교체하여 사용하니 latency가 가장 낮았던 ViSTA-SLAM만큼 가벼워졌고, VGGT frontend를 사용했던 Ours보다 성능은 낮아졌지만, 그래도 기존 baseline method들보다 훨씬 좋은 성능을 보임

Ablation Study

- Impact of Submap Configuration

w	$\phi = 1$	$\phi = 2$	$\phi = 4$	$\phi = 8$
4	0.043	0.041	—	—
8	0.035	0.035	0.035	—
16	0.031	0.032	0.031	0.030
32	0.033	0.032	0.034	0.033

- ϕ : number of overlap frame between submaps
- w : submap size

- 하나의 frame만을 공유할 때는($\phi = 1$), submap 정렬이 하나의 multi-view local pose estimation에만 의존하여, 불안정한 최적화를 유발함. (따라서 $\phi = 2$ 을 기본값으로 사용)
 - 더 큰 submap은 local multi-view estimation을 향상시키지만, graph에서 submap node의 개수는 줄게되는데, 이는 global constraints를 약화시킴. (따라서 $w = 16$ 을 기본값으로 사용)
- Effectiveness of Graph Constraints

	w/o Backend	w/o LC	w/o $\mathbf{e}^{2v}$	w/o $\mathbf{e}^{anch}$	w/o $\mathbf{e}^{br}$	w/o $\mathbf{e}^{tie}$	w/o $\mathbf{e}^{sc}$	Ours (full)
$\phi = 0$	0.124	0.061	0.101	0.083	0.116	0.040	0.048	0.032
$\phi = 2$	0.124	0.037	0.021	0.020	0.063	0.027	0.031	0.020

More Details

- residual weights

$$(w^{2v}, w^{br}, w^{anch}, w^{sc}, w^{tie}) = (1.0, 1.0, 50.0, 50.0, 0.2)$$

Residual	weights	chess	fire	heads	office	pumpkin	kitchen	stairs	Avg.
e^{sc}, e^{anch}	$w^{sc}, w^{anch} = 1$	0.030	0.024	0.027	0.027	0.021	0.022	0.028	0.025
	$w^{sc}, w^{anch} = 2$	0.024	0.022	0.027	0.026	0.021	0.021	0.025	0.024
	$w^{sc}, w^{anch} = 10$	0.018	0.019	0.027	0.025	0.021	0.019	0.020	0.021
	$w^{sc}, w^{anch} = 50$	0.017	0.018	0.026	0.024	0.022	0.016	0.019	0.020
e^{tie}	$w^{tie} = 0.2$	0.017	0.018	0.026	0.024	0.022	0.016	0.019	0.020
	$w^{tie} = 0.5$	0.018	0.018	0.026	0.024	0.021	0.015	0.019	0.020
	$w^{tie} = 1.0$	0.020	0.020	0.026	0.023	0.020	0.015	0.019	0.020
	$w^{tie} = 2.0$	0.025	0.023	0.027	0.024	0.020	0.015	0.019	0.022

scale 관련 weight가 작아질수록, tie weight이 증가할수록 성능이 감소함

7-Scenes

More Details

- residual weights

	Method	chess	fire	heads	office	pumpkin	kitchen	stairs	Avg.
overlap ($\phi = 2$)	Tracking only	0.158	0.090	0.101	0.145	0.166	0.114	0.096	0.124
	w/o Loop closure	0.020	0.024	0.029	0.075	0.036	0.050	0.023	0.037
	w/o $\mathbf{e}^{\text{sc}}$	0.045	0.032	0.027	0.030	0.023	0.022	0.036	0.031
	w/o $\mathbf{e}^{2v}$	0.020	0.018	0.026	0.024	0.023	0.016	0.019	0.021
	w/o $\mathbf{e}^{\text{anch}}$	0.019	0.018	0.026	0.024	0.022	0.015	0.018	0.020
	w/o $\mathbf{e}^{\text{br}}$	0.046	0.048	0.035	0.110	0.121	0.051	0.027	0.063
	w/o $\mathbf{e}^{\text{tie}}$	0.018	0.020	0.026	0.035	0.033	0.034	0.021	0.027
	w/o $\mathcal{E}^{v2s}$	0.158	0.090	0.101	0.145	0.166	0.114	0.096	0.124
	w/o $\mathcal{E}^{s2s}$	0.017	0.023	0.027	0.037	0.030	0.052	0.025	0.030
	Ours (full)	0.017	0.018	0.026	0.024	0.022	0.016	0.019	0.020
Non-overlap	Tracking only	0.158	0.090	0.101	0.145	0.166	0.114	0.096	0.124
	w/o Loop closure	0.106	0.045	0.035	0.074	0.075	0.059	0.033	0.061
	w/o $\mathbf{e}^{\text{sc}}$	0.117	0.056	0.032	0.033	0.037	0.033	0.030	0.048
	w/o $\mathbf{e}^{2v}$	0.157	0.064	0.059	0.065	0.162	0.111	0.085	0.101
	w/o $\mathbf{e}^{\text{anch}}$	0.178	0.185	0.070	0.047	0.020	0.042	0.040	0.083
	w/o $\mathbf{e}^{\text{br}}$	0.216	0.098	0.075	0.138	0.125	0.112	0.045	0.116
	w/o $\mathbf{e}^{\text{tie}}$	0.106	0.043	0.041	0.024	0.020	0.033	0.015	0.040
	w/o $\mathcal{E}^{v2s}$	0.158	0.090	0.101	0.145	0.166	0.114	0.096	0.124
	w/o $\mathcal{E}^{s2s}$	0.092	0.029	0.033	0.027	0.034	0.045	0.025	0.041
	Ours (full)	0.066	0.029	0.043	0.027	0.023	0.019	0.015	0.032

7-Scenes

View-to-Submap, Submap-to-Submap edge set에 대한 ablation 평가도 진행

More Details

- run time

	Frontend	Backend				
Component	Two-view inference	Submap node initialization	Descriptor extraction	Graph construction	Optimization	Multi-view inference
Latency[ms]	197	167	16	36	645	933

- frontend는 7-Scenes에서 대략 25 FPS를 달성하면서 two-view estimation을 진행
- backend는 새 프레임이 충분히 누적되었을 때만 동작
 - default configuration에서는 14개의 새로운 keyframe이 쌓였을 때 backend 과정 진행
- frontend와 backend는 분리된 thread에서 동작하기 때문에, low-latency tracking 성능을 유지하면서 backend refinement가 비동기적으로 처리됨

More Details

- run time (vs VGGT-SLAM 2.0)

	Frontend	Backend				
Component	Two-view inference	Submap node initialization	Descriptor extraction	Graph construction	Optimization	Multi-view inference
Latency[ms]	197	167	16	36	645	933

Sum: 1994

Stage	total time per submap (ms)
VGGT Inference	1248 ms
Keyframe Detection	176 ms
Loop Closure Detection	128 ms
Backend Optimization	8 ms
Semantics (optional)	368 ms

⇒ 하지만 backend는 새로운 keyframe 이 어느정도 모였을 때만 일어나기 때문에, 이를 고려했을 때 low-latency라고 볼 수 있음

VGGT-SLAM 2.0

Sum: 1928

More Details

- Other Backbone

Method		chess	fire	heads	office	pumpkin	kitchen	stairs	Avg.
Calib.	DROID-SLAM [36]	0.018	0.027	0.021	0.041	0.025	0.016	0.017	0.024
	MASt3R-SLAM [26]	0.082	0.030	0.024	0.052	0.050	0.044	0.027	0.044
Uncalib.	CUT3R [40]	0.514	0.110	0.197	0.430	0.346	0.202	0.385	0.312
	SLAM3R [21]	0.131	0.044	0.040	0.058	0.100	0.064	0.116	0.079
	MASt3R-SLAM* [26]	0.090	0.058	0.039	0.072	0.084	0.062	0.071	0.068
	VG GT-SLAM [22]	0.039	0.024	0.041	0.032	0.050	0.034	0.042	0.037
	ViSTA-SLAM [47]	0.075	0.035	0.030	0.064	0.065	0.041	0.036	0.049
	Ours + STA	0.030	0.031	0.031	0.023	0.029	0.024	0.018	0.027
	Ours	0.017	0.018	0.026	0.024	0.022	0.016	0.019	0.020

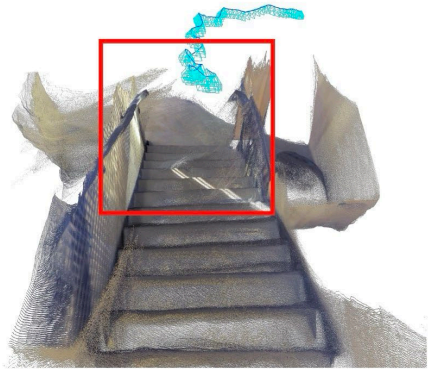
7-Scenes

Method		360	desk	desk2	floor	plant	room	rpy	teddy	xyz	Avg
Calib.	ORB-SLAM3 [3]	×	0.017	0.210	×	0.034	×	×	×	0.009	N/A
	DeepV2D [35]	0.243	0.166	0.379	1.653	0.203	0.246	0.105	0.316	0.064	0.375
	DeepFactors [4]	0.159	0.170	0.253	0.169	0.305	0.364	0.043	0.601	0.035	0.233
	DPV-SLAM [19]	0.112	0.018	0.029	0.057	0.021	0.330	0.030	0.084	0.010	0.076
	DPV-SLAM++ [19]	0.132	0.018	0.029	0.050	0.022	0.096	0.032	0.098	0.010	0.054
	GO-SLAM [49]	0.089	0.016	0.028	0.025	0.026	0.052	0.019	0.048	0.010	0.035
	DROID-SLAM [36]	0.111	0.018	0.042	0.021	0.016	0.049	0.026	0.048	0.012	0.038
	MASt3R-SLAM [26]	0.049	0.016	0.024	0.025	0.020	0.061	0.027	0.041	0.009	0.030
Uncalib.	CUT3R [40]	0.174	0.592	0.546	0.662	0.467	0.911	0.051	0.845	0.129	0.486
	SLAM3R [21]	0.211	0.861	0.967	0.790	0.755	1.013	0.063	0.986	0.185	0.648
	MASt3R-SLAM* [26]	0.070	0.032	0.055	0.056	0.035	0.118	0.041	0.116	0.020	0.060
	VG GT-SLAM [22]	0.063	0.031	0.048	0.152	0.023	0.133	0.038	0.039	0.020	0.061
	ViSTA-SLAM [47]	0.104	0.030	0.030	0.070	0.052	0.067	0.023	0.080	0.015	0.052
	Ours + STA	0.065	0.016	0.022	0.035	0.045	0.041	0.022	0.034	0.012	0.032
	Ours	0.067	0.018	0.022	0.034	0.029	0.056	0.021	0.031	0.013	0.032

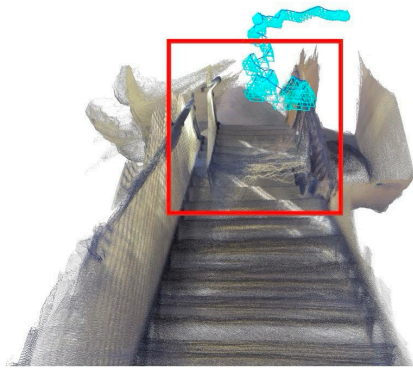
TUM RGB-D

More Details

- Qualitative Result



Result with PGO

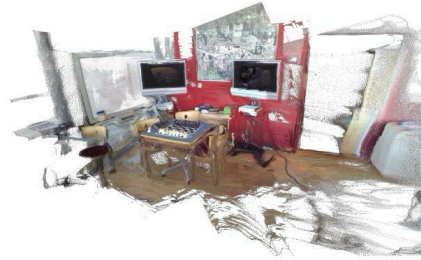


Result without PGO

7-Scenes (stairs)

- PGO를 쓰니 misalignment가 해결됨

7-Scenes chess



7-Scenes fire



7-Scenes pumpkin



TUM RGB-D desk



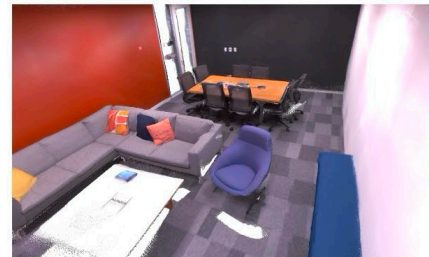
TUM RGB-D xyz



TUM RGB-D floor



Replica office3



Replica room0



Replica room1



Conclusion

- UniSim-SLAM은 two-view, multi-view constraint를 multi-level factor graph를 이용해 하나로 묶은 feed-forward SLAM system
- 이를 통해 low-latency로 consistent global pose estimation이 가능하게 됨
- SoTA 모델들 대비 높은 성능(camera pose estimation, 3d reconstruction)을 보임

내 생각 (왜 잘된 것 같은가?)

- 기존 multi-view method(e.g. VGGT-SLAM)가 two-view method(e.g. MAST3R-SLAM) 보다 높은 성능을 보였었는데, two-view가 기존 multi-view의 어떤 점을 보완했는가?
 - 기존 multi-view method 대비 다른점은 frontend를 two-view로 쓰고, backend에서 View-to-Submap을 통해 two-view, multi-view constraint를 하나로 묶는다는 점
 - 7-Scenes에서 진행한 ablation study에서 two-view residual은 chess sequence에서밖에 성능 향상을 보이지 않음
 - 따라서 해당 two-view residual은 scale inconsistency에 강건함을 제공하기는 하지만, 성능 기여에 있어서 더 큰 비중은 View-to-Submap residual에 있는듯함
- ablation study에서 loop closure가 없어도 기존 SoTA 모델들에 comparable한 성능을 보였다고 했는데, 해당 ablation study는 어떤 데이터셋으로 진행됐었고, long-sequence에서도 비슷한 점을 보였는가?
 - 7-Scenes에서 진행되었고, 해당 데이터셋의 시퀀스 대부분은 방 1개에서 촬영
 - 따라서 long-sequence에 대한 평가는 없었고 loop closure의 필요성이 극대화되는 구역에서의 추가 평가가 필요해 보임
- TUM RGB-D floor 시퀀스가 planar structure dominant하여 scale recovery를 위한 geometric cue가 부족해, 기존 feed-forward model들은 scale drift를 발생시킨다고 했는데, UniSim-SLAM은 이러한 상황에서 어떻게 강건하게 동작할 수 있었는가?
 - 해당 시퀀스에서 성능이 VGGT-SLAM 2.0 대비 2배나 좋았음 (MASt3R-SLAM도 잘나온걸 보니 two-view inference가 이런 부분에 강건함을 주는?)
- 따로 outdoor dataset에 대한 평가가 있었는가?
 - 없었음
- 따로 추가 학습을 하는건가?
 - feed-forward model에 대한 extra training은 없음
 - 새로운 multi-level factor graph에 대한 real-time optimization만 수행되는 듯 함